



University of Engineering and Technology
Lahore, new Campus

OOP Lab Project Report

Project Name: Hospital Management System (HMS)

Representors (Group of Three):

1. Mirza Zain Ali Baig – 2025(S)-SE-20
2. Numan Amdad – 2025(S)-SE-19
3. Arslan Mohsin – 2025(S)-SE-07

Under the guidance of

‘Madam Zoha’

1. Problem Statement:

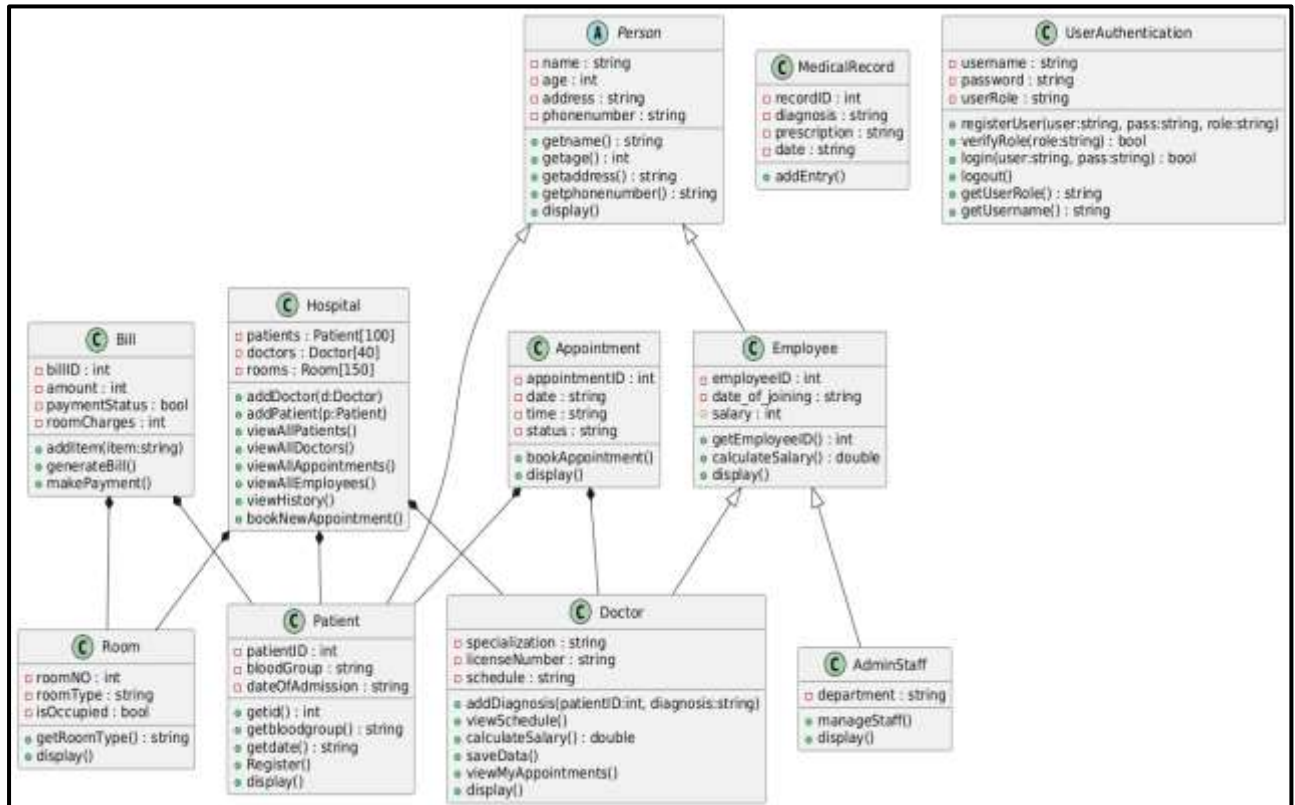
- The project aims to develop a robust, secure, and user-friendly hospital management system.
- The system is designed to manage the day-to-day operations of a modern hospital.
- It is built using Object-Oriented Programming (OOP) principles.
- The system will automate patient registration.

2. Solution:

- Develop a menu-driven, object-oriented system to manage hospital operations.
- Create separate classes for Patient, Doctor, Appointment, Billing, and Medical Records.
- Use encapsulation to protect sensitive patient and billing data.
- Use inheritance to create different types of users (Admin, Doctor, Receptionist).
- Apply polymorphism to allow different user roles to perform actions in their own way.
- Implement file handling to store patient records, appointments, and bills permanently.
- Add a login system to ensure authorized access and security.

- Automate patient registration and doctor scheduling through class methods.
 - Enable appointment booking and cancellation.
 - Generate bills automatically based on services used.
 - Provide search and update options for patient and doctor records.
-
- It will manage doctor scheduling.
 - It will handle appointment management.
 - The system will provide billing services.
 - It will maintain medical records and data.

3. UML Diagram



Classes and their Core Functions:

1. Person

- getName() → Returns the name of the person.
- getAge() → Returns the age.
- getAddress() → Returns the address.
- getPhoneNumber() → Returns the phone number.
- display() → Displays personal details.

2. Patient (inherits Person)

- `getID()` → Returns the patient's unique ID.
 - `getBloodGroup()` → Returns blood group of the patient.
 - `getDate()` → Returns registration or admission date.
 - `register()` → Registers a new patient into the system.
 - `display()` → Shows patient details.
-

3. Doctor (inherits Person)

- `addDiagnosis(patientID, diagnosis)` → Adds medical diagnosis for a patient.
 - `viewSchedule()` → Shows the doctor's daily schedule.
 - `calculateSalary()` → Calculates doctor's salary.
 - `saveData()` → Saves doctor's information.
 - `viewMyAppointments()` → Displays all appointments assigned to the doctor.
 - `display()` → Shows doctor details.
-

4. Employee (inherits Person)

- `getEmployeeID()` → Returns employee's unique ID.
 - `calculateSalary()` → Calculates employee salary.
 - `display()` → Displays employee details.
-

5. AdminStaff (inherits Employee)

- `manageStaff()` → Manages hospital employees.
 - `display()` → Displays admin staff details.
-

6. Appointment

- `bookAppointment()` → Books an appointment between doctor and patient.
 - `display()` → Shows appointment details.
-

7. MedicalRecord

- `addEntry()` → Adds a new medical record entry for a patient.
-

8. Hospital

- `addDoctor(Doctor d)` → Adds a new doctor to the hospital.
 - `addPatient(Patient p)` → Adds a new patient.
 - `viewAllPatients()` → Displays all registered patients.
 - `viewAllDoctors()` → Displays all doctors.
 - `viewAllAppointments()` → Displays all appointments.
 - `viewAllEmployees()` → Displays all employees.
 - `viewHistory()` → Shows hospital activity history.
 - `bookNewAppointment()` → Creates a new appointment.
-

9. Bill

- `addItem(item)` → Adds charges to the bill.

- generateBill() → Generates total bill.
 - makePayment() → Processes bill payment.
-

10. Room

- getRoomType() → Returns the type of room (ICU, General, etc.).
 - display() → Displays room information.
-

11. UserAuthentication

- registerUser(username, password, role) → Registers a new user.
- verifyRole(role) → Checks user role.
- login(username, password) → Logs a user into the system.
- logout() → Logs out the user.
- getUserRole() → Returns the user's role.
- getUsername() → Returns the logged-in username

Relationships:

1. Inheritance

These classes inherit from Person:

- Patient inherits Person
- Doctor inherits Person
- Employee inherits Person

These classes inherit from Employee:

- AdminStaff inherits Employee
-

2. Association (Has-a relationships)

Hospital has:

- Hospital has Patient[100]
- Hospital has Doctor[40]
- Hospital has Room[150]

(These are strong associations because hospital stores their objects.)

Appointment is associated with:

- Appointment is linked to Patient
- Appointment is linked to Doctor

(An appointment exists between a patient and a doctor.)

Doctor is associated with:

- Doctor is associated with Appointment
- Doctor is associated with MedicalRecord

(Doctor creates diagnosis and handles patient records.)

Patient is associated with:

- Patient is associated with Appointment

- Patient is associated with Bill
- Patient is associated with Room

(Patient books appointments, receives bills, and is assigned rooms.)

Bill is associated with:

- Bill is linked to Patient
- Bill is linked to Room

(Bill includes room charges and patient charges.)

UserAuthentication is associated with:

- UserAuthentication is associated with AdminStaff
- UserAuthentication is associated with Doctor
- UserAuthentication is associated with Patient

(All users log in through authentication.)

3. Aggregation / Composition

- Hospital → Patient, Doctor, Room
 - Aggregation (Hospital manages them but they can exist independently)
- Patient → Bill
 - Composition (Bill cannot exist without a patient)
- Patient → MedicalRecord
 - Composition (Medical record belongs to one patient)

Operator Overloading:

Used operator Overloading like:

1. == Operator
2. << operator for output data
3. Much more...

Sample Code:

```
bool operator==(const Patient& other) const {  
    return this->patientID == other.patientID;  
}  
  
// Friend function for output  
friend ostream& operator<<(ostream& os, const Patient& p) {  
    os << "Patient ID: " << p.patientID << ", Name: " << p.name << ", Age: " << p.age  
    << ", Blood Group: " << p.bloodGroup << ", Admission Date: " << p.dateOfAdmission;  
    return os;  
}
```

File Handling:

used file handling that saved the data to associative files

Sample Files where data will save:

```
const string patientFile = "patientFile.txt";
const string doctorFile = "doctorFile.txt";
const string appointmentFile = "appointmentFile.txt";
const string roomFile = "roomFile.txt";
const string medicalRecordFile = "medicalRecordFile.txt";
const string billFile = "billFile.txt";
const string adminStaffFile = "adminStaffFile.txt";
const string authFile = "userAuthFile.txt";
```

Sample Outputs:

Admin Login:

```
===== HOSPITAL MANAGEMENT SYSTEM =====
=====
1- Admin Login
2- Doctor Login
3- Operator Login
4- Exit
=====
Enter your choice: 1

=== ADMIN LOGIN ===
Username: admin
Password: admin123
Admin login successful!
```

Admin Menu:

```
===== ADMIN MENU =====
```

- 1- Add new doctor
- 2- Add new employee
- 3- Add new operator
- 4- Hospital Details
- 5- History
- 6- Logout

Add new doctor

```
=== Add New Doctor ===  
Enter Doctor ID: 1  
Enter Name: Ali  
Enter Age: 45  
Enter Address: Lahore  
Enter Phone: 03258709422  
Enter Specialization: Heart  
Enter License Number: wj9e  
Enter Schedule (e.g., Mon-Fri 9AM-5PM): Monday-Saturday  
Enter Date of Joining: 12 jan  
Enter Salary: 400000  
Doctor added to hospital system successfully.  
Doctor added successfully!
```

History:

```
===== System Logs =====
1. Patient Records

===== All Patients =====
No patient records found!
2. Doctor Records

===== All Doctors =====
1. ID: 1, Name: Dr. Ali, Specialization: Heart, License: wj9e
=====

3. Appointment Records

===== All Appointments =====
No appointments scheduled yet!
=====

4. Employee Records

===== All Employees =====
No employee records found!
=====
```